

Code-Layout, Readability and Reusability

Date	24 June 2026
Team ID	SWTID-2026-3249
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the OptiCrop codebase in terms of its structure, readability, and reusability. The project follows a clean Flask application layout with separated concerns for model training, backend routing, and presentation, ensuring the codebase is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation used consistently across app.py and train.py
2	Proper File Structure	Files and folders are logically organized	Yes	Separated into templates/, static/css/, app.py, train.py and model.pkl
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. MODEL_PATH, features, crop_name, formatted_crop
4	Function / Method Names	Functions are descriptively named	Yes	Flask routes named home(), about(), predict() reflect their purpose
5	Code Comments	Inline and block comments explain logic	Yes	Key steps commented, e.g. model loading, feature formatting, prediction step
6	Modular Design	Code is split into reusable functions/modules	Yes	Training pipeline (train.py) is separate from the serving layer (app.py)
7	No Redundant Code	Duplicate or unused code is removed	Yes	Single prediction route handles both form display and inference
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except blocks around model loading and prediction with user-friendly fallback messages

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	layout.html (base template)	Jinja2 / HTML5	Extended by index.html, predict.html, about.html	High
2	Model loading utility	Python / Pickle	Loaded once at app startup and reused for every /predict call	High
3	style.css	CSS3 / Bootstrap	Shared styling across all pages	High
4	Feature-formatting logic	NumPy	Converts form inputs into the 2D array expected by the model	Medium

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
5	train.py preprocessing pipeline	Pandas / Scikit-learn	Reusable for retraining on updated datasets	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of ML pipeline and Flask app layers
Readability	5	Descriptive names and comments make logic easy to follow
Reusability	4	Templates and preprocessing logic can be reused for new datasets
Documentation / Comments	4	Key functions documented; docstrings could be expanded
Overall Score	4.5	Well-structured, readable, and maintainable Flask + ML codebase